

Cryptography and Embedded System Security

CRAESS_I

doc. Xiaolu Hou, PhD.

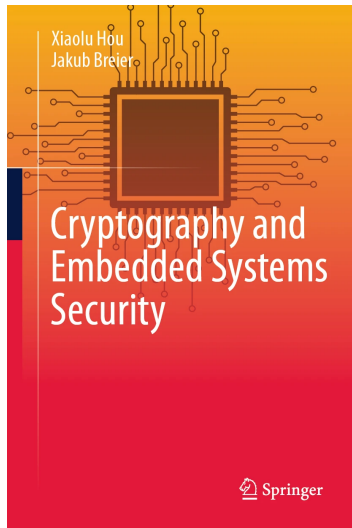
FIIT, STU
xiaolu.hou @ stuba.sk

Course Outline

- Abstract algebra and number theory
- Introduction to cryptography
- Symmetric block ciphers and their implementations
- RSA, RSA signatures, and their implementations
- Probability theory and introduction to SCA
- SPA and non-profiled DPA
- Profiled DPA
- SCA countermeasures
- FA on RSA and countermeasures
- FA on symmetric block ciphers
- FA countermeasures for symmetric block cipher
- Practical aspects of physical attacks
 - Invited speaker: Dr. Jakub Breier, Senior security manager, TTControl GmbH

Recommended reading

- Textbook
 - Sections 5.1.1, 5.1.2



Lecture Outline

- Differential Fault Analysis
- Statistical Fault Analysis
- Other Fault Attacks
- Assignment 6

Fault attacks on symmetric block ciphers

- The specifications of round functions and key schedules are public (Kerckhoffs' principle)
- The master key, and hence also the round keys, is secret.
- We also assume that the same master key is used throughout the attack, and that the attacker's goal is usually to recover certain round key(s).
- The methodologies that we will discuss can be applied to many unprotected implementations of symmetric block ciphers.
- Fault attacks normally aim to recover the last or first round key(s), and then use the inverse key schedule to recover the master key.
 - DES: any round key $\rightarrow$ 48 bits of master key
 - AES: any round key $\rightarrow$ master key
 - PRESENT: any round key $\rightarrow$ 64 bits of master key, brute force the rest or recover another round key

Remark

- We will see two attacks on AES
 - Differential fault analysis attack
 - Statistical fault analysis attack
- There are many more fault attack methods

Fault mask

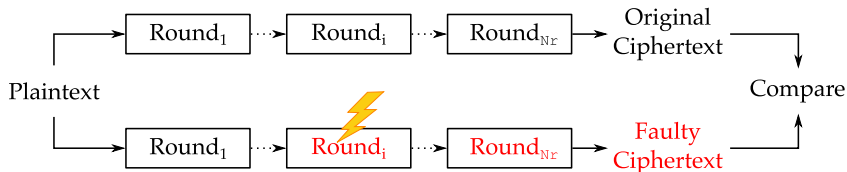
- If a fault injected into an intermediate value x results in a faulty value x' , then
- We refer to $\varepsilon := x \oplus x'$ as the *fault mask*, which represents the change in the faulted value.

FA on symmetric block ciphers

- Differential Fault Analysis
- Statistical Fault Analysis
- Other Fault Attacks
- Assignment 6

Attack methodology

- Differential Fault Analysis (DFA) was first introduced by Biham et al.¹ in 1997.
- It has been studied by numerous researchers in different settings and is one of the most popular fault attack analysis methods for symmetric block ciphers.
- DFA considers a fault injected into an intermediate state of the cipher, usually in the last few rounds.
- The difference between the correct and faulty ciphertexts is then analyzed to recover the round key(s).



¹Biham, E., & Shamir, A. (1997, August). Differential fault analysis of secret key cryptosystems. In Annual international cryptology conference (pp. 513-525). Springer, Berlin, Heidelberg.

Difference distribution table

Definition

For an Sbox $SB : \mathbb{F}_2^{\omega_1} \rightarrow \mathbb{F}_2^{\omega_2}$, the (*extended*) *difference distribution table (DDT)* of SB is a 2-dimensional table T of size $2^{\omega_2} \times (2^{\omega_1} - 1)$ such that for any $0 < \delta < 2^{\omega_1}$ and $0 \leq \Delta < 2^{\omega_2}$, the entry of T at the Δ th row and δ th column is given by

$$T[\Delta, \delta] = \{ \mathbf{a} \mid \mathbf{a} \in \mathbb{F}_2^{\omega_1}, SB(\mathbf{a} \oplus \delta) \oplus SB(\mathbf{a}) = \Delta \}.$$

Here we identify elements of $\mathbb{F}_2^{\omega_1}$ and $\mathbb{F}_2^{\omega_2}$ with their binary representations as integers when indexing the rows and columns of the table. We refer to δ as the *input difference*, and Δ as the *output difference*.

Difference distribution table

Definition

$$T[\Delta, \delta] = \{ \mathbf{a} \mid \mathbf{a} \in \mathbb{F}_2^{\omega_1}, \text{SB}(\mathbf{a} \oplus \delta) \oplus \text{SB}(\mathbf{a}) = \Delta \}.$$

We refer to δ as the *input difference*, and Δ as the *output difference*.

Example (DDT of PRESENT Sbox)

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

If the input is 9 and the input difference is $\delta = 3$, then the output difference is given by

$$\text{SB}_{\text{PRESENT}}(9 \oplus 3) \oplus \text{SB}_{\text{PRESENT}}(9) = \text{SB}_{\text{PRESENT}}(\text{A}) \oplus 1110 = 1111 \oplus 1110 = 0001 = 1.$$

Thus 9 is in $T[1, 3]$. Similarly, 7 is in $T[4, 3]$

$$\text{SB}_{\text{PRESENT}}(7 \oplus 3) \oplus \text{SB}_{\text{PRESENT}}(7) = \text{SB}_{\text{PRESENT}}(4) \oplus 1101 = 1001 \oplus 1101 = 0100 = 4.$$

DDT of PRESENT Sbox

$\Delta \backslash \delta$	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1			9A		36		078F				5E		1C		24BD
2						8E	34		09	5F		1D	67AB	2C	
3	CDEF	46	12					3B		0A			58	79	
4			47		8D				35AC		0B		2F		169E
5		CDEF		0145						2389		67AB			
6		9B	CDEF	37		06	25		18						4A
7	67AB		03	8C				5D				2E	49	1F	
8						17	AD		6F	4E	2389	0C		5B	
9	0145			9D	BE			2A			7C	3F		68	
A		02	56	BF	9C					7D	1A	48	3E		
B			8B		27	35AC		169E			4F		0D		
C		8A		26	0145	9F	BC		7E					3D	
D	2389	57			AF			4C		1B	6D			0E	
E		13		AE					24BD	6C		59			078F
F						24BD	169E	078F							35AC

Table: Difference distribution table for PRESENT Sbox, where the row corresponding to output difference $\Delta = 0$ is omitted since it is empty

DDT – Example

Example

x	0	1	2	3	4	5	6	7
$SB(x)$	4	7	0	5	2	6	3	1

$\Delta \backslash \delta$	1	2	3	4	5	6	7
1		46	03	15	27		
2	67	13			05	24	
3	01		47	26		35	
4	45	02		37			16
5	23		56		14		07
6				04	36	17	25
7		57	12			06	34

$$SB(0) \oplus SB(0 \oplus 3) = 4 \oplus SB(3) = 4 \oplus 5 = 1$$

How DFA works on a simple example

- Let us consider the AND operation, which takes inputs $a, b \in \mathbb{F}_2$ and outputs

$$c = a \& b.$$

- All possible values of a, b, c are given by

a	b	$c = a \& b$
0	0	0
0	1	0
1	0	0
1	1	1

- Suppose the attacker can observe the output c , while a and b are unknown.
- The attacker injects a fault in b by flipping it.
- By comparing the correct and faulty outputs, how does the attacker recover the value of a ?

How DFA works on a simple example

- All possible values of a, b, c are given by

a	b	$c = a \ \& \ b$
0	0	0
0	1	0
1	0	0
1	1	1

- Suppose the output c can be observed by the attacker and a, b are unknown.
- The attacker injects a fault in b by flipping it.
- If the output c stays the same, then $a = 0$; otherwise $a = 1$.

How DFA works on PRESENT Sbox

- SB: PRESENT Sbox
- Let $\mathbf{a} \in \mathbb{F}_2^4$, $\mathbf{b} \in \mathbb{F}_2^4$ be fixed secret values
- Define

$$\begin{aligned} f : \mathbb{F}_2^4 &\rightarrow \mathbb{F}_2^4 \\ \mathbf{x} &\mapsto \text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \mathbf{b}. \end{aligned}$$

- We will show how to recover the values of $\mathbf{a}$ and $\mathbf{b}$ with DFA
- Attack assumption
 - Fault location: input of f
 - Fault model: bit flip
 - Fault mask: $\varepsilon \in \mathbb{F}_2^4$ s.t. $\mathbf{x}' = \mathbf{x} \oplus \varepsilon$
 - Attacker knowledge: the Sbox design, the input $\mathbf{x}$, the correct and faulty outputs of f , and the fault mask.
 - Attacker goal: recover values of $\mathbf{a}$ and $\mathbf{b}$
 - Attacker can repeat the computation with the same input (not chosen by attacker)

How DFA works on PRESENT Sbox

$$\begin{aligned} f : \mathbb{F}_2^4 &\rightarrow \mathbb{F}_2^4 \\ \mathbf{x} &\mapsto \text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \mathbf{b}. \end{aligned}$$

- Attack assumption
 - Fault location: input of f
 - Fault model: bit flip
 - Fault mask: $\varepsilon \in \mathbb{F}_2^4$ s.t. $\mathbf{x}' = \mathbf{x} \oplus \varepsilon$
 - Attacker knowledge: Sbox design, inputs and outputs of f , fault mask
 - Attacker goal: recover values of $\mathbf{a}$ and $\mathbf{b}$
 - Attacker can repeat the computation with the same input (not chosen by attacker)
- Attack steps:
 - Compute DDT of the Sbox, T
 - Inject fault
 - Reduce the guesses for $\mathbf{a}$ using knowledge of the fault mask, the input, and outputs.
 - Reduce the guesses for $\mathbf{b}$ using the guesses for $\mathbf{a}$ and knowledge of the correct input and output.

How DFA works on PRESENT Sbox

$$\begin{aligned} f : \mathbb{F}_2^4 &\rightarrow \mathbb{F}_2^4 \\ x &\mapsto \text{SB}(x \oplus a) \oplus b. \end{aligned}$$

Attack steps:

- Compute DDT of the Sbox, T
- Inject fault
- Reduce the guesses for a using knowledge of the fault mask, the input, and the outputs.

- Let Δ denote the difference between the correct and faulty output, then

$$\Delta = (\text{SB}(x \oplus a) \oplus b) \oplus (\text{SB}(x' \oplus a) \oplus b) = \text{SB}(x \oplus a) \oplus \text{SB}(x' \oplus a) = \text{SB}(x \oplus a) \oplus \text{SB}(x \oplus a \oplus \varepsilon)$$

- Thus the value $x \oplus a$ is in the entry of T corresponding to input difference $\delta = \varepsilon$ and output difference Δ
- Reduce the guesses for b using the guesses for a and knowledge of the correct input and output.

How DFA works on PRESENT Sbox – Example

$$\begin{aligned} f : \mathbb{F}_2^4 &\rightarrow \mathbb{F}_2^4 \\ x &\mapsto \text{SB}(x \oplus a) \oplus b. \end{aligned}$$

Reduce the guesses for a using knowledge of the fault mask, the input, and the outputs.

- Let Δ denote the difference between the correct and faulty output, then
$$\Delta = (\text{SB}(x \oplus a) \oplus b) \oplus (\text{SB}(x' \oplus a) \oplus b) = \text{SB}(x \oplus a) \oplus \text{SB}(x' \oplus a) = \text{SB}(x \oplus a) \oplus \text{SB}(x \oplus a \oplus \varepsilon)$$
- Thus the value $x \oplus a$ is in the entry of T corresponding to input difference $\delta = \varepsilon$ and output difference Δ

Example

- Suppose the attacker fixes the input to be $x = 0$ and they know that the correct output of f is 0
- When the attacker injects fault in x with fault mask $\varepsilon_1 = 3$, they get a faulty output 1, which gives $\Delta_1 = ?$

How DFA works on PRESENT Sbox – Example

$\Delta \backslash \delta$	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1			9A		36		078F				5E		1C		24BD
2						8E	34		09	5f		1D	67AB	2C	
3	CDEF	46	12					3B		0A			58	79	
4			47		8D				35AC		0B		2F		169E
5		CDEF		0145						2389		67AB			
6		9B	CDEF	37		06	25		18						4A
7	67AB		03	8C				5D				2E	49	1F	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

Example

- Input: $x = 0$; correct output: 0
- Fault mask: $\varepsilon_1 = 3$; faulty output: 1, which gives $\Delta_1 = 0 \oplus 1 = 1$.
- $x \oplus a$ is in the entry corresponding to input difference $\delta = 3$ and output difference 1 of T
- The possible values for $x \oplus a$ are given by?

How DFA works on PRESENT Sbox – Example

$\Delta \backslash \delta$	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	9A		36		078F		5E		1C		24BD				
2	8E				34	09		5f	1D		67AB	2C			
3	CDEF	46	12	3B				0A		58		79			
4	47		8D		35AC		0B		2F		169E				
5	CDEF		0145		2389		67AB								
6	9B		CDEF	37	06	25	18		4A						
7	67AB	03		8C	5D				2E		49	1F			
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

Example

- Input: $x = 0$; correct output: 0
- Fault mask: $\varepsilon_1 = 3$; faulty output: 1 $\rightarrow$ the possible values for $x \oplus a$ are 9 and A
- Fault mask $\varepsilon_2 = 2$; faulty output: 6 $\rightarrow \Delta_2 = 6$
- $x \oplus a$ is in the entry of T corresponding to input difference $\delta = 2$ and output difference 6
- Possible values of $x \oplus a$ are ?

How DFA works on PRESENT Sbox – Example

$$\begin{aligned} f : \mathbb{F}_2^4 &\rightarrow \mathbb{F}_2^4 \\ \mathbf{x} &\mapsto \text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \mathbf{b}. \end{aligned}$$

Reduce the guesses for $\mathbf{b}$ using the guesses for $\mathbf{a}$ and knowledge of the correct input and output.

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

Example

- Input: $\mathbf{x} = 0$; correct output: 0
- fault mask: $\varepsilon_1 = 3$; faulty output: 1 $\rightarrow$ the possible values for $\mathbf{x} \oplus \mathbf{a}$ are 9 and A
- fault mask $\varepsilon_2 = 2$; faulty output: 6 $\rightarrow$ possible values for $\mathbf{x} \oplus \mathbf{a}$ are 9 and B
- $\mathbf{a} = 9$, $\mathbf{b} = \text{E}$

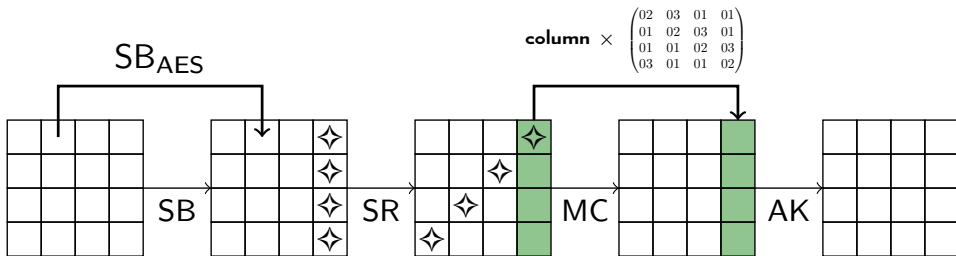
How many faults are needed

$\Delta \backslash \epsilon$	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
1			9a		36		078f				5e		1c		24bd
2						8e	34		09	5f		1d	67ab	2c	
3	cdef	46	12					3b		0a			58	79	
4			47		8d				35ac		0b		2f		169e
5		cdef		0145						2389		67ab			
6		9b	cdef	37		06	25		18						4a
7	67ab		03	8c				5d				2e	49	1f	
8						17	ad		6f	4e	2389	0c		5b	
9	0145			9d	be			2a			7c	3f		68	
a		02	56	bf	9c					7d	1a	48	3e		
b			8b		27	35ac		169e			4f		0d		
c		8a		26	0145	9f	bc		7e					3d	
d	2389	57			af			4c		1b	6d			0e	
e		13		ae					24bd	6c		59			078f
f						24bd	169e	078f							35ac

- Chosen fault mask: 2 faults may suffice (e.g. masks 3 and 5)
- Random fault mask: at most 4 faults are needed

AES encryption

- An initial AddRoundKey
- Round function for $N_r - 1$ rounds: SubBytes, ShiftRows, MixColumns, AddRoundKey
- Last round, round N_r : SubBytes, ShiftRows, AddRoundKey
- AddRoundKey is bitwise XOR with the round key
- SubBytes is the application of 8-bit Sboxes.
- ShiftRows permutes the bytes
- MixColumns is a function on 32-bit values (four bytes).



Fault propagation in AES

- Recall that the AES cipher state can be represented as a four-by-four matrix of bytes:

$$\begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix}.$$

- For visualization purposes, let us represent these bytes as squares.
- Suppose a fault is injected at the beginning of one round (except for the last round) in byte s_{00} .
- Then the fault propagation in this round can be represented by

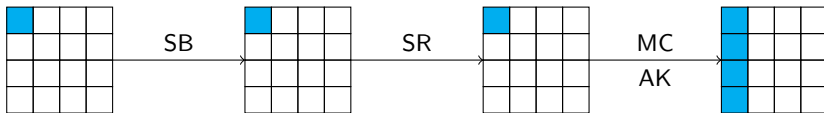


Figure: Blue squares correspond to bytes that can be affected by the fault.

Fault propagation in AES

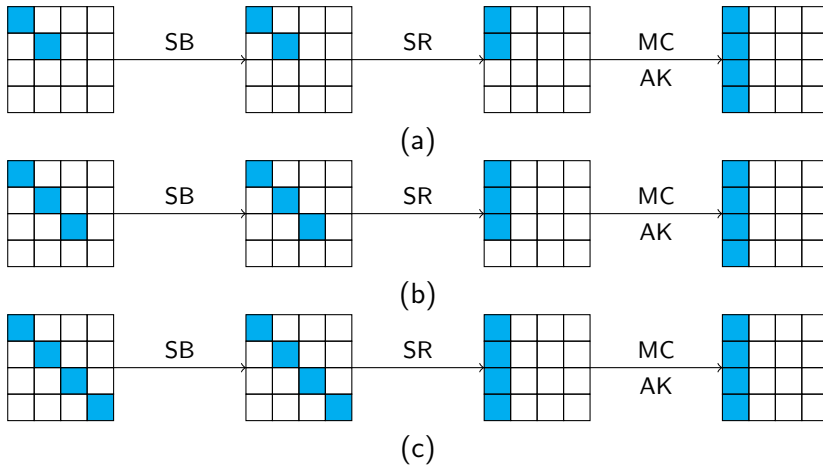


Figure: Visual illustration of how the fault propagates when a fault is injected at the beginning of one AES round in bytes (a) s_{00}, s_{11} , (b) s_{00}, s_{11}, s_{22} , and (c) $s_{00}, s_{11}, s_{22}, s_{33}$. Blue squares correspond to bytes that can be affected by the fault.

Fault at end of round 7

- Let us refer to the bytes $s_{00}, s_{11}, s_{22}, s_{33}$ as a *diagonal* of the AES state.
- We consider a fault attack where a random byte fault is injected in the diagonal of the AES state at the end of round 7.
- By the above discussion, we know that at the end of round 8, the whole first column might be affected by the fault.
- Similarly, we can study the fault propagation in round 9.
- Recall that MixColumns multiplies one column by the following matrix

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

Fault propagation in round 9

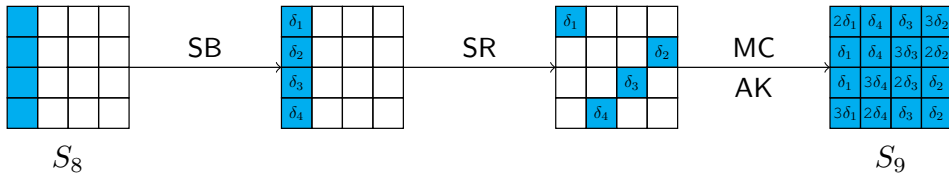


Figure: Visual illustration of fault propagation in the 9th round of AES when the fault was injected in the diagonal $s_{00}, s_{11}, s_{22}, s_{33}$ of the AES cipher state at the end of round 7. δ_i ($i = 1, 2, 3, 4$) denote the differences between the four correct and faulty bytes in the first column of the cipher state after SubBytes in round 9.

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot$$

Notations

- S_9 : the cipher state at the end of round nine
- c : the correct ciphertext
- K_{10} : the last round key

$$S_9 = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix}, c = \begin{pmatrix} c_{00} & c_{01} & c_{02} & c_{03} \\ c_{10} & c_{11} & c_{12} & c_{13} \\ c_{20} & c_{21} & c_{22} & c_{23} \\ c_{30} & c_{31} & c_{32} & c_{33} \end{pmatrix}, K_{10} = \begin{pmatrix} k_{00} & k_{01} & k_{02} & k_{03} \\ k_{10} & k_{11} & k_{12} & k_{13} \\ k_{20} & k_{21} & k_{22} & k_{23} \\ k_{30} & k_{31} & k_{32} & k_{33} \end{pmatrix}.$$

- c' : faulty ciphertext

$$c' = \begin{pmatrix} c'_{00} & c'_{01} & c'_{02} & c'_{03} \\ c'_{10} & c'_{11} & c'_{12} & c'_{13} \\ c'_{20} & c'_{21} & c'_{22} & c'_{23} \\ c'_{30} & c'_{31} & c'_{32} & c'_{33} \end{pmatrix}.$$

- Round 10: SubBytes, ShiftRows, and AddRoundKey.

$$c_{00} = \text{SB}_{\text{AES}}(a_{00}) \oplus k_{00}, \quad c_{13} = \text{SB}_{\text{AES}}(a_{10}) \oplus k_{13},$$

$$c_{22} = \text{SB}_{\text{AES}}(a_{20}) \oplus k_{22}, \quad c_{31} = \text{SB}_{\text{AES}}(a_{30}) \oplus k_{31}.$$

- Then

$$a_{00} = \text{SB}_{\text{AES}}^{-1}(c_{00} \oplus k_{00})$$

$$a_{10} = \text{SB}_{\text{AES}}^{-1}(c_{13} \oplus k_{13})$$

$$a_{20} = \text{SB}_{\text{AES}}^{-1}(c_{22} \oplus k_{22})$$

$$a_{30} = \text{SB}_{\text{AES}}^{-1}(c_{31} \oplus k_{31}).$$

- Similarly

$$a'_{00} = \text{SB}_{\text{AES}}^{-1}(c'_{00} \oplus k_{00})$$

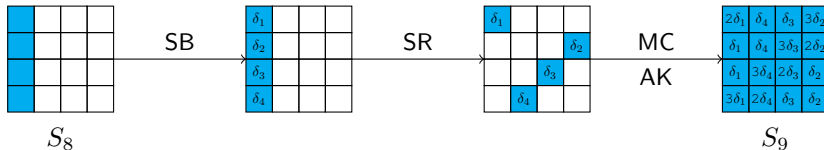
$$a'_{10} = \text{SB}_{\text{AES}}^{-1}(c'_{13} \oplus k_{13})$$

$$a'_{20} = \text{SB}_{\text{AES}}^{-1}(c'_{22} \oplus k_{22})$$

$$a'_{30} = \text{SB}_{\text{AES}}^{-1}(c'_{31} \oplus k_{31}).$$

- Let $\delta = \delta_1$ and by observing the first column of S_9 , we have

$$\begin{aligned}
 2\delta &= a_{00} \oplus a'_{00} = \text{SB}_{\text{AES}}^{-1}(c_{00} \oplus k_{00}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{00} \oplus k_{00}) \\
 \delta &= a_{10} \oplus a'_{10} = \text{SB}_{\text{AES}}^{-1}(c_{13} \oplus k_{13}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{13} \oplus k_{13}) \\
 \delta &= a_{20} \oplus a'_{20} = \text{SB}_{\text{AES}}^{-1}(c_{22} \oplus k_{22}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{22} \oplus k_{22}) \\
 3\delta &= a_{30} \oplus a'_{30} = \text{SB}_{\text{AES}}^{-1}(c_{31} \oplus k_{31}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{31} \oplus k_{31}).
 \end{aligned}$$



Key recovery

$$\begin{aligned}2\delta &= a_{00} \oplus a'_{00} = \text{SB}_{\text{AES}}^{-1}(c_{00} \oplus k_{00}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{00} \oplus k_{00}) \\ \delta &= a_{10} \oplus a'_{10} = \text{SB}_{\text{AES}}^{-1}(c_{13} \oplus k_{13}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{13} \oplus k_{13}) \\ \delta &= a_{20} \oplus a'_{20} = \text{SB}_{\text{AES}}^{-1}(c_{22} \oplus k_{22}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{22} \oplus k_{22}) \\ 3\delta &= a_{30} \oplus a'_{30} = \text{SB}_{\text{AES}}^{-1}(c_{31} \oplus k_{31}) \oplus \text{SB}_{\text{AES}}^{-1}(c'_{31} \oplus k_{31}).\end{aligned}$$

- For each value of δ , the possible values for $(k_{00}, k_{13}, k_{22}, k_{31})$ are restricted
- $a_{00} = \text{SB}_{\text{AES}}^{-1}(c_{00} \oplus k_{00})$ can be considered as an AES Sbox input that corresponds to input difference 2δ and output difference $c_{00} \oplus c'_{00}$
- $a_{10} = \text{SB}_{\text{AES}}^{-1}(c_{13} \oplus k_{13})$ is an AES Sbox input that gives output difference $c_{13} \oplus c'_{13}$ when the input difference is δ
- On average¹, the key hypotheses for $(k_{00}, k_{13}, k_{22}, k_{31})$ can be reduced to 2^8

¹Saha, D., Mukhopadhyay, D., & RoyChowdhury, D. (2009). A diagonal fault attack on the advanced encryption standard. Cryptology ePrint Archive.

Diagonal DFA

- In this attack, we assume that the attacker knows
 - The fault location: diagonal of cipher state at the end of round 7
 - Fault model: random byte
 - Output of AES: correct and faulty ciphertext
- Since the attack is on the diagonal of the cipher state, it is also called the *diagonal DFA*.

Other diagonals

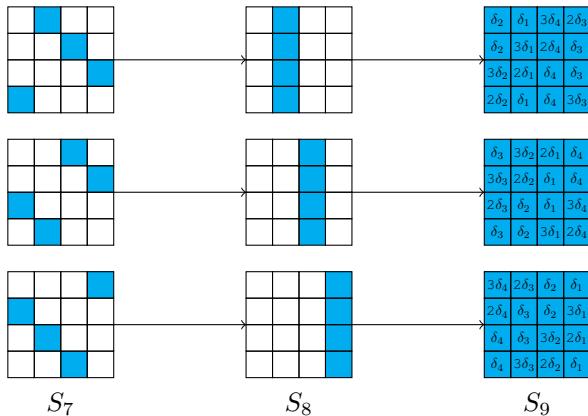


Figure: Fault propagation for random byte fault injected in the “diagonals” of the cipher state at the end of round 7. S_i denotes the cipher state at the end of the i th round

FA on symmetric block ciphers

- Differential Fault Analysis
- Statistical Fault Analysis
- Other Fault Attacks
- Assignment 6

Fault distribution table

- Consider fault models that change an intermediate value x to x' .
- Model these two values as random variables X and X' .
- Based on the fault properties, we can draw a table with probabilities for the value x to be changed to x' , i.e. $P(X' = x' | X = x)$ – *fault distribution table*

Example

- Let us consider the case when x is just one bit.
- stuck-at-0: changes x to 0 with probability 1.
- bit flip fault: changes x to $x \oplus 1$ with probability 1.
- random fault: changes x to $x \oplus 1$ with probability 0.5.

x'

x		0	1
0		1	0
1		1	0

stuck-at-0

		0	1
0		0	1
1		1	0

bit flip

		0	1
0		0.5	0.5
1		0.5	0.5

random

Assumption

- Statistical Fault Analysis (SFA)¹ assumes no knowledge of plaintext or correct ciphertext for the attacker.
- Only knowledge of the faulty ciphertext and a non-uniform fault model is required.
- We say that the fault model is *non-uniform* if

$$P(X' = x' | X = x) \neq \frac{1}{2^b}$$

for some x and x' , where b is the maximum bit length of x .

¹Fuhr, T., Jaulmes, É., Lomné, V., & Thillard, A. (2013, August). Fault attacks on AES with faulty ciphertexts only. In 2013 Workshop on Fault Diagnosis and Tolerance in Cryptography (pp. 108-118). IEEE.

Non-uniform fault model – Example

Example

- Let us consider the case where x is a single bit.
- In this case, the bit length of x is 1, and a fault model is non-uniform if

$$P(X' = x' | X = x) \neq 0.5$$

for some x and x' .

- stuck-at-0 and bit flip fault models are non-uniform.

		x'				x'					
		0	1			0	1				
x	0	1	0		0	0	1		0	0.5	0.5
	1	1	0		1	1	0		1	0.5	0.5
		stuck-at-0				bit flip				random	

Fault injection at the end of AES round 9

- S_9 : the cipher state at the end of round nine
- c : the correct ciphertext
- K_{10} : the last round key

$$S_9 = \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix}, c = \begin{pmatrix} c_{00} & c_{01} & c_{02} & c_{03} \\ c_{10} & c_{11} & c_{12} & c_{13} \\ c_{20} & c_{21} & c_{22} & c_{23} \\ c_{30} & c_{31} & c_{32} & c_{33} \end{pmatrix}, K_{10} = \begin{pmatrix} k_{00} & k_{01} & k_{02} & k_{03} \\ k_{10} & k_{11} & k_{12} & k_{13} \\ k_{20} & k_{21} & k_{22} & k_{23} \\ k_{30} & k_{31} & k_{32} & k_{33} \end{pmatrix}.$$

- A fault is injected in s_{00} according to a non-uniform fault model.
- S_{00} : random variable corresponding to s_{00}
- S'_{00} : random variable corresponding to faulty value of S_{00} , s'_{00}
- Attacker knowledge: the fault location and the fault distribution table, i.e. the probabilities

$$P(S'_{00} = s'_{00} | S_{00} = s_{00})$$

- Goal of the attacker: recover k_{00} .

SFA – Example

Example

Let us consider a stuck-at-0 fault model, then

$$P(S'_{00} = s'_{00} | S_{00} = s_{00}) = \begin{cases} 1 & s'_{00} = 00 \\ 0 & \text{Otherwise} \end{cases},$$

for all $s_{00} \in \mathbb{F}_2^8$.

In this case, one faulty ciphertext is enough to recover k_{00} . Since the attacker knows that the faulty value s'_{00} is always 00, the attacker can recover k_{00} by computing

$$k_{00} = c'_{00} \oplus \text{SB}_{\text{AES}}(00) = c'_{00} \oplus 63.$$

Probability of a faulty value

We assume S_{00} follows a uniform distribution, i.e.

$$P(S_{00} = s_{00}) = \frac{1}{256} \quad \forall s_{00} \in \mathbb{F}_2^8.$$

Then

$$\begin{aligned} P(S'_{00} = s'_{00}) &= \sum_{s_{00}=0}^{255} P(S'_{00} = s'_{00} | S_{00} = s_{00}) P(S_{00} = s_{00}) \\ &= \frac{1}{256} \sum_{s_{00}=0}^{255} P(S'_{00} = s'_{00} | S_{00} = s_{00}). \end{aligned}$$

Recall – Fault injection at the end of AES round 9

- S_9 : the cipher state at the end of round nine
- c : the correct ciphertext
- K_{10} : the last round key

$$S_9 = \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix}, c = \begin{pmatrix} c_{00} & c_{01} & c_{02} & c_{03} \\ c_{10} & c_{11} & c_{12} & c_{13} \\ c_{20} & c_{21} & c_{22} & c_{23} \\ c_{30} & c_{31} & c_{32} & c_{33} \end{pmatrix}, K_{10} = \begin{pmatrix} k_{00} & k_{01} & k_{02} & k_{03} \\ k_{10} & k_{11} & k_{12} & k_{13} \\ k_{20} & k_{21} & k_{22} & k_{23} \\ k_{30} & k_{31} & k_{32} & k_{33} \end{pmatrix}.$$

$$c_{00} = \text{SB}_{\text{AES}}(s_{00}) \oplus k_{00} \implies s_{00} = \text{SB}_{\text{AES}}^{-1}(c_{00} \oplus k_{00}).$$

Attack steps

- Injects fault in s_{00}
- Collects a set of m faulty ciphertexts $\{c'^1, c'^2, \dots, c'^m\}$.
- Let $\hat{k}_{00}$ denote a key hypothesis for k_{00} .
- Then for each c'^i , we can compute a hypothetical value for s'_{00} , denoted $\hat{s}_{00}^i$, as follows:

$$\hat{s}_{00}^i = \text{SB}_{\text{AES}}^{-1}(c'^i \oplus \hat{k}_{00}).$$

- Last round: SubBytes, ShiftRows, AddRoundKey

Attack steps

$$\begin{aligned} P(S'_{00} = s'_{00}) &= \sum_{s_{00}=0}^{255} P(S'_{00} = s'_{00} | S_{00} = s_{00}) P(S_{00} = s_{00}) \\ &= \frac{1}{256} \sum_{s_{00}=0}^{255} P(S'_{00} = s'_{00} | S_{00} = s_{00}). \end{aligned}$$

- The probability that the faulty value of s_{00} in the i th encryption equals $\hat{s}_{00}^i$ can be computed from the fault distribution table using the formula above:

$$P(S'_{00} = \hat{s}_{00}^i) = \frac{1}{256} \sum_{s_{00}=0}^{255} P(S'_{00} = \hat{s}_{00}^i | S_{00} = s_{00}).$$

Attack steps

- Assuming the fault injections are independent, define $\ell(\hat{k}_{00})$ to be the probability that the faulty value of s_{00} in the i th encryption equals the hypothetical value $\hat{s}_{00}^i$ for all i , i.e.

$$\ell(\hat{k}_{00}) := \prod_{i=1}^m P(S'_{00} = \hat{s}_{00}^i). \quad (1)$$

- Then the correct key can be found using the *maximum likelihood* approach, namely

$$k_{00} = \arg \max_{\hat{k}_{00}} \ell(\hat{k}_{00}).$$

Attack results

- It was shown¹ that with high probability, the correct key byte can be recovered with only a few faults.
- The same method can be used to recover the other bytes of K_{10} .
- Each byte can be recovered in parallel; hence, the number of faults needed to recover the full round key depends on the number of bytes that can be faulted by one fault injection.

¹Fuhr, T., Jaulmes, É., Lomné, V., & Thillard, A. (2013, August). Fault attacks on AES with faulty ciphertexts only. In 2013 Workshop on Fault Diagnosis and Tolerance in Cryptography (pp. 108-118). IEEE.

FA on symmetric block ciphers

- Differential Fault Analysis
- Statistical Fault Analysis
- Other Fault Attacks
- Assignment 6

Ineffective Fault Analysis

- Clavier, C. (2007, September). Secret external encodings do not prevent transient fault analysis. In International Workshop on Cryptographic Hardware and Embedded Systems (pp. 181-194). Springer, Berlin, Heidelberg.
- Faults that do not change the intermediate values are exploited.
- Those faults are called *ineffective faults*.
- Normally a particular fault model is assumed, e.g. a stuck-at-0 fault model.

Statistical Ineffective Fault Attack (SIFA)

- Dobraunig, C., Eichlseder, M., Korak, T., Mangard, S., Mendel, F., & Primas, R. (2018). SIFA: exploiting ineffective fault inductions on symmetric cryptography. IACR Transactions on Cryptographic Hardware and Embedded Systems, 547-572.
- A non-uniform fault model is assumed and the attack exploits ineffective faults.
- The dependency between the fault induction being ineffective and the data that is processed is exploited.
- Different from SFA, SIFA does not require each fault to be successful, but the attack requires repeated plaintext, and knowledge of the correct ciphertext (or whether each ciphertext is correct or not).
- The fault injection is the same as for SFA
- In the original paper the authors provide a detailed theoretical analysis of the number of ciphertexts needed and extensive experimental results.

Persistent Fault Analysis (PFA)

- Zhang, Fan, Xiaoxuan Lou, Xinjie Zhao, Shivam Bhasin, Wei He, Ruyi Ding, Samiya Qureshi, and Kui Ren. Persistent fault analysis on block ciphers. IACR Transactions on Cryptographic Hardware and Embedded Systems (2018): 150-172.
- Faults are injected into memory, the Sbox lookup table
- Knowledge of ciphertext only

Algebraic Fault Analysis (AFA)

- Courtois, N. T., Jackson, K., & Ware, D. (2010). Fault-algebraic attacks on inner rounds of DES. In E-Smart'10 Proceedings: The Future of Digital Security Technologies. Strategies Telecom and Multimedia.
- Similar to DFA, AFA exploits differences between correct and faulty ciphertexts.
- DFA – manual analysis
- AFA – expresses cryptographic algorithm in the form of algebraic equations and utilizes SAT solver¹ to recover the key.

¹A SAT solver solves Boolean satisfiability problems. It takes a Boolean logic formula and checks if there is a solution satisfying the formula.

Collision Fault Analysis

- Blömer, J., & Seifert, J. P. (2003, January). Fault based cryptanalysis of the advanced encryption standard (AES). In International Conference on Financial Cryptography (pp. 162-181). Springer, Berlin, Heidelberg.
- Injects fault in the earlier rounds of a block cipher implementation.
- Then the attacker records the faulty ciphertext and finds plaintext that produces the same ciphertext, but without fault.
- Further analysis using those plaintexts can recover the round key.
- If the fault only changes one bit or one byte of the intermediate value, the attacker can try different plaintexts that only differ at one bit or one byte.

Fault Sensitivity Analysis

- Li, Y., Sakiyama, K., Gomisawa, S., Fukunaga, T., Takahashi, J., & Ohta, K. (2010, August). Fault sensitivity analysis. In International workshop on cryptographic hardware and embedded systems (pp. 320-334). Springer, Berlin, Heidelberg.
- Exploits the sensitivity of a device to faults
- The attack analyzes when a faulty output begins to exhibit detectable characteristics and uses this information to recover the secret key.
- No knowledge of faulty ciphertext is required for the attack.

FA on symmetric block ciphers

- Differential Fault Analysis
- Statistical Fault Analysis
- Other Fault Attacks
- Assignment 6

Binary code

- Let n be a positive integer
- We focus on the vector space $\mathbb{F}_2^n$

Definition

- $\mathbf{w} = w_0w_1 \dots w_{n-1} \in \mathbb{F}_2^n$ is called a *binary word of length n* .
- A nonempty subset $C \subseteq \mathbb{F}_2^n$ is called a *binary code of length n* .
- An element of a binary code C is called a *codeword of C* .
- The cardinality of C is called the *size of C* .
- A code of length n and size M is called a *binary (n, M) -code*.

Example

- $C = \{00, 11\}$ is a binary $(2, 2)$ -code.
- $C = \{010, 001, 110, 111\}$ is a binary $(3, 4)$ -code.

Question 1

Question

Let $C = \{ 0001, 0100, 1100 \}$. What are the length and size of C ?

Solution

C has length 4 and size 3.

Hamming distance

Definition

For any $\mathbf{v}, \mathbf{u} \in \mathbb{F}_2^n$, the *Hamming distance* between $\mathbf{v}$ and $\mathbf{u}$, denoted $\text{dis}(\mathbf{v}, \mathbf{u})$, is defined as follows

$$\text{dis}(\mathbf{v}, \mathbf{u}) = \sum_{i=0}^{n-1} \text{dis}(v_i, u_i), \text{ where } \text{dis}(v_i, u_i) = \begin{cases} 1 & \text{if } v_i \neq u_i \\ 0 & \text{if } v_i = u_i \end{cases}. \quad (2)$$

Example

$\text{dis}(001, 111) = 2$ and $\text{dis}(00000, 10101) = 3$.

Binary code

Definition

Let $C \subseteq \mathbb{F}_2^n$ be a binary code containing at least two codewords. The (*minimum*) *distance* of C , denoted $\text{dis}(C)$, is defined by

$$\text{dis}(C) = \min \{ \text{dis}(\mathbf{c}_1, \mathbf{c}_2) \mid \mathbf{c}_1, \mathbf{c}_2 \in C, \mathbf{c}_1 \neq \mathbf{c}_2 \}.$$

Definition

A binary code of length n , size M and distance d is called a *binary (n, M, d) -code*.

Example

Let $C = \{ 0011, 1101, 1000 \}$, we can calculate that

$$\text{dis}(0011, 1101) = 3, \quad \text{dis}(0011, 1000) = 3, \quad \text{dis}(1101, 1000) = 2.$$

Thus C is a binary $(4, 3, 2)$ -code

Question 2

Question

Let $C = \{00000, 11100, 10101, 01010\}$. What is the distance of C ?

Solution

$$\begin{aligned} \text{dis}(00000, 11100) &= 3, & \text{dis}(00000, 10101) &= 3, & \text{dis}(00000, 01010) &= 2 \\ \text{dis}(11100, 10101) &= 2, & \text{dis}(11100, 01010) &= 3, & \text{dis}(10101, 01010) &= 5 \end{aligned}$$

Thus C has distance 2.

Error-detecting code

When the value of a bit is changed, we say that the bit is flipped.

Definition

A binary code C is said to be *k-error-detecting* for a positive integer k if for any $c \in C$, whenever at least 1 but at most k bits of c are flipped, the resulting word is not a codeword in C . If C is k -error detecting but not $(k + 1)$ -error detecting, then we say that C is *exactly k-error detecting*.

Theorem

A binary (n, M, d) -code C is k -error detecting if and only if $d \geq k + 1$. Equivalently, C is exactly $(d - 1)$ -error detecting.

Question 3

Question

Let $C = \{00000, 11100, 10101, 01010\}$, C is exactly ?-error detecting.

Solution

From the previous question we know that C has distance 2. By the above theorem, C is exactly 1-error detecting.

Encoding and decoding

- Let us consider binary (n, M, d) -codes with $M = 2^k$ for some positive integer k .
- When a binary code is used for transmitting information, every information word $\mathbf{u} \in \mathbb{F}_2^k$ is assigned a unique codeword $\mathbf{c}(\mathbf{u}) \in C$.
- We say that $\mathbf{u}$ is *encoded* as $\mathbf{c}(\mathbf{u})$.
- Suppose Alice wants to send information $\mathbf{u}$ to Bob using C .
- Alice sends the codeword $\mathbf{c}(\mathbf{u})$ to Bob.
- Due to transmission noise, Bob might receive a word $\mathbf{x} \in \mathbb{F}_2^n$ not equal to $\mathbf{c}(\mathbf{u})$.
- Thus, we need to define a *decoding rule* that allows Bob to find $\mathbf{u}$ given $\mathbf{x}$.

Minimum distance decoding

- We are interested in a *minimum distance decoding rule*, which specifies that after receiving $\mathbf{x}$, Bob computes¹

$$\mathbf{c}_x = \arg \min_{\mathbf{c}} \{ \text{dis}(\mathbf{x}, \mathbf{c}) \mid \mathbf{c} \in C \},$$

i.e.

$$\text{dis}(\mathbf{c}_x, \mathbf{x}) = \min_{\mathbf{c}} \{ \text{dis}(\mathbf{x}, \mathbf{c}) \mid \mathbf{c} \in C \}.$$

- If more than one codeword is identified as $\mathbf{c}_x$, then there are two options.
 - *Incomplete decoding rule*: Bob asks Alice for another transmission.
 - *Complete decoding rule*: Bob randomly selects one codeword.

¹Recall difference between sets defined in week 1.

Encoding and decoding – Example

Example

Let $C = \{0000, 0111, 1110, 1111\}$. We use C to encode information words $u \in \mathbb{F}_2^2$ using the following encoding:

$$c(00) = 0000, \quad c(01) = 0111, \quad c(10) = 1110, \quad c(11) = 1111.$$

Suppose Alice was sending information 00 with codeword 0000 to Bob. Due to an error during the transmission, Bob received 0001. By the minimum distance decoding rule, Bob computes the distances between 0001 and codewords in C .

$$\text{dis}(0001, 0000) = 1, \quad \text{dis}(0001, 0111) = 2, \quad \text{dis}(0001, 1110) = 4, \quad \text{dis}(0001, 1111) = 3$$

Thus $c_{0001} = 0000$ and Bob gets the correct information 00.

Question 4

Question

Continuing from the previous example, if Bob receives 0011, what is the decoding result under the minimum distance decoding rule?

Solution

By the minimum distance decoding rule, Bob computes the distances between 0011 and codewords in C .

$$\text{dis}(0011, 0000) = 2, \text{dis}(0011, 0111) = 1, \text{dis}(0011, 1110) = 3, \text{dis}(0011, 1111) = 2$$

The decoding result is $c_{0011} = 0111$.

Error-correcting code

Definition

A binary code C is said to be *k -error correcting* if, under the incomplete decoding rule, minimum distance decoding outputs the correct codeword whenever k or fewer bits are flipped. If C is k -error correcting but not $(k + 1)$ -error correcting, then we say that C is *exactly k -error correcting*.

Example

Let $C = \{ 000, 111 \}$.

- If 000 was sent and 1 bit flip occurred, the received word $\{ 001, 010, 100 \}$ will be decoded to 000.
- If 111 was sent and 1 bit flip occurred, the received word $\{ 110, 011, 101 \}$ will be decoded to 111.
- If 000 was sent and 011 was received, the decoding result will be 111.

Thus C is exactly 1-error correcting.

Error-correcting code

Theorem

A binary (n, M, d) -code C is k -error correcting if and only if $d \geq 2k + 1$; equivalently, C is an exactly $\lfloor (d - 1)/2 \rfloor$ -error correcting code.

Example (Repetition code)

Let

$$C = \{ 00 \dots 00, 11 \dots 11 \} \subseteq \mathbb{F}_2^n.$$

C is a binary $(n, 2, n)$ -code. C is called the *binary n -repetition code*. By the two theorems above, C is exactly $(n - 1)$ -error detecting and exactly $\lfloor (n - 1)/2 \rfloor$ -error correcting.

Question 5

Question

Let C be the binary 5-repetition code.

- a) What are the codewords in C ?
- b) C is exactly ?-error detecting.
- c) C is exactly ?-error correcting.

Solution

- a) $C = \{ 00000, 11111 \}$
- b) C is exactly 4-error detecting.
- c) C is exactly 2-error correcting.

Anticode

Definition

Let C be a binary (n, M, d) -code. We define the *maximum distance* of C to be

$$\text{maxdis}(C) := \max \{ \text{dis}(\mathbf{c}_1, \mathbf{c}_2) \mid \mathbf{c}_1, \mathbf{c}_2 \in C \}.$$

If $\text{maxdis}(C) = \delta$, C is called a *binary (n, M, d, δ) -anticode*.

- We note that $\delta \geq d$.
- Any binary code is also a binary anticode. However, the notion of binary anticode captures the maximum distance of a code.

Example

- $C = \{01, 10\}$ is a binary $(2, 2, 2, 2)$ -anticode.
- $C = \{001, 011, 111\}$ is a binary $(3, 3, 1, 2)$ -anticode.
- An n -repetition code is a binary $(n, 2, n, n)$ -anticode.

Encoding XOR

- Next, let us look at the computation of XOR between two bits:

$$\begin{aligned}\text{XOR} : \mathbb{F}_2 \times \mathbb{F}_2 &\rightarrow \mathbb{F}_2 \\ (a, b) &\mapsto a \oplus b.\end{aligned}$$

- We would like to encode the inputs and the output of this operation.
- The operation will be implemented with a lookup table.
- We further require that the all-zero vector $\mathbf{0}$ is not a codeword, and that the output $\mathbf{0}$ indicates that an error has occurred.

Encoding XOR – Example

Example

- Suppose we choose to use $\{01, 10\}$, a binary $(2, 2, 2, 2)$ -anticode.
- Let 01 be the codeword for 0 and 10 be the codeword for 1.
- For decoding, we map each codeword to the corresponding value (0 or 1), but when the input is not a codeword, we output 00 to indicate an error.
- Then the lookup table is as follows:

	00	01	10	11
00	00	00	00	00
01	00	01	10	00
10	00	10	01	00
11	00	00	00	00

- For example, when the input is 0 and 1, the corresponding codewords are 01 and 10. The output should be $0 \oplus 1 = 1$, which corresponds to the codeword 10. Thus the entry for row 01 and column 10 is 10.

Encoding XOR – Example

Example

- We choose to use $\{01, 10\}$, a binary $(2, 2, 2, 2)$ -anticode.
- Let 01 be the codeword for 0 and 10 be the codeword for 1.

	00	01	10	11
00	00	00	00	00
01	00	01	10	00
10	00	10	01	00
11	00	00	00	00

- Any 1-bit flip will be detected
 - If the fault is injected into input 01, then a 1-bit flip gives either 00 or 11, both of which give output 00.
 - If a 1-bit flip is injected into input 10, then we obtain either 00 or 11, and the output is again 00.

Encoding XOR – Example

Example

- We choose to use $\{01, 10\}$, a binary $(2, 2, 2, 2)$ -anticode.
- Let 01 be the codeword for 0 and 10 be the codeword for 1.

	00	01	10	11
00	00	00	00	00
01	00	01	10	00
10	00	10	01	00
11	00	00	00	00

- A 2-bit flip will be undetected.
 - For example, suppose we would like to compute $0 \oplus 0$.
 - Then the inputs for the table lookup will be 01 and 01, the output will be 01, which corresponds to 0.
 - If a 2-bit flip is injected in the first input, we get 10 and 01 for table lookup.
 - The result will be 10.
 - Such a fault will not be detected and can change the output of the operation.

Encoding XOR

- In general, if the minimum distance of the binary code used is d , then the code is an exactly $(d - 1)$ -error detecting code.
- Consequently, the XOR lookup design described above can detect up to $(d - 1)$ -bit flips.

Question 6

Question

Suppose we choose to use $\{010, 101\}$ for encoding the XOR operation inputs and outputs. Take 010 to be the codeword for 0 and 101 to be the codeword for 1. Use a design similar to the one above.

a) What is the lookup table for implementing XOR?

Solution

	000	001	010	011	100	101	110	111
000	000	000	000	000	000	000	000	000
001	000	000	000	000	000	000	000	000
010	000	000	010	000	000	101	000	000
011	000	000	000	000	000	000	000	000
100	000	000	000	000	000	000	000	000
101	000	000	101	000	000	010	000	000
110	000	000	000	000	000	000	000	000
111	000	000	000	000	000	000	000	000

Question 6

Question

Suppose we choose to use $\{010, 101\}$ for encoding the XOR operation inputs and outputs. Take 010 to be the codeword for 0 and 101 to be the codeword for 1. Use a design similar to the one above.

- b) Suppose an m -bit flip fault is injected in one of the inputs. What is the maximum value of m such that the fault injection can be detected?

Solution

- b) The code has minimum distance 3 and hence is exactly 2-error detecting. The table lookup implementation can detect up to 2-bit flip faults.

Lookup table with error-correcting code

- Now, let us consider a different way of designing the lookup table.
- Instead of using one specific word to indicate an error, we follow the minimum distance complete decoding rule.
- If the binary code used has minimum distance d , then it is an exactly $\lfloor (d-1)/2 \rfloor$ -error correcting code.
- Thus, such a lookup table design allows us to correct up to $\lfloor (d-1)/2 \rfloor$ -bit flips.

Question 7

Question

For example, let us consider the 3-repetition code $\{000, 111\}$, which has minimum distance 3. Using this code, we can correct errors caused by 1-bit flip attacks. Let 000 be the codeword for 0 and 111 be the codeword for 1. What is the lookup table for implementing XOR?

Solution

	000	001	010	011	100	101	110	111
000	000	000	000	111	000	111	111	111
001	000	000	000	111	000	111	111	111
010	000	000	000	111	000	111	111	111
011	111	111	111	000	111	000	000	000
100	000	000	000	111	000	111	111	111
101	111	111	111	000	111	000	000	000
110	111	111	111	000	111	000	000	000
111	111	111	111	000	111	000	000	000

Question 7

For example, let us consider the 3-repetition code $\{000, 111\}$, which has minimum distance 3. Using this code, we can correct errors caused by 1-bit flip attacks. Let 000 be the codeword for 0 and 111 be the codeword for 1.

	000	001	010	011	100	101	110	111
000	000	000	000	111	000	111	111	111
001	000	000	000	111	000	111	111	111
010	000	000	000	111	000	111	111	111
011	111	111	111	000	111	000	000	000
100	000	000	000	111	000	111	111	111
101	111	111	111	000	111	000	000	000
110	111	111	111	000	111	000	000	000
111	111	111	111	000	111	000	000	000

- From the table, we can see that any 1-bit flip will be corrected.
- However, if there are more bit flips, the faulty output might be corrected to a wrong codeword.

Future Lectures

- 8th May, online, invited lecture
- 12th May, in person, Lecture 11
 - Consultation about final exam questions
- Final exam: 18th May, 10:30-12:30, Room 1.40